Practice question on RNN

1. Consider an encoder-decoder model for sequence to sequence learning as used in tasks like translation. Let $\mathbf{h}_1, \ldots, \mathbf{h}_n$ denote encoder states, and $\mathbf{z}_1, \ldots, \mathbf{z}_t$ denote the decoder states. Let y_t denote the token output at time t. Provide the equations for computing the distribution over the next token y_{t+1} using the notations below and any intermediate variables that you need:

   - Output(a, b) computes the output vector in terms of input a, b. You need to figure out what a, b should be.
   - Softmax(o) computes softmax using vector o
   - Attn(a, b) denotes the un-normalized attention score in terms of inputs a and b. You need to figure out what a and b should be.
   - $E(y)$ returns the embedding vector of a token y
   - D-RNN(x,z) computes the updated decoder state given an input x and current state z.

   The first step has been shown for you to give an idea of the format of the answers. Each step is specified as a one line sentence description and a formula.

   Step 1

   Lookup embedding of token y_t:

   $$r = E(y_t)$$

   .

   Step 2

   Step 3

   Step 4

   Step 5

   Step 6

   Step 7

..4

2. Consider a seq2seq model using an encoder-decoder network. Suppose, we are considering a setting (example in speech to text) where the attention variables along time need to be monotonic. How can you exploit this information to generate a better attention distribution at t-th decoding step? Assume the encoder RNN states are $\mathbf{h}^i$ for $i = 1, \ldots, n$, the decoder RNN states $\mathbf{z}^t$ for $t = 1, \ldots, m$. For your reference the standard method of computing attention is

$$\text{Attn}(\mathbf{z}_{t-1}, \mathbf{h}_i) = A_{ti}, \quad \alpha_{ti} = \frac{\exp(A_{ti})}{\sum_p \exp(A_{tp})} \tag{1}$$

where $\alpha_{t1}, \ldots, \alpha_{tn}$ denotes the attention over encoder states at time t and sum to 1.

..3

3. Consider a transformer with the following key, query and value matrices:

$$K = \begin{bmatrix} 0 & 1 & 0 \\ 1.1 & 0 & 0 \end{bmatrix}$$

$$Q = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 1 \end{bmatrix}$$

$$V = \begin{bmatrix} 0.6 & -0.1 \\ 0.2 & 3 \end{bmatrix}$$

Calculate the final output of the self attention layer (Take $d_k = 1$) [3 marks] Take ln 2 as 0.7 and ln 3 as 1.1.

$$Output = \begin{bmatrix} w & x \\ y & z \end{bmatrix}$$

4. In this question, assume that the transformer attention is just $(Q \times K^T)V$, that is the softmax is missing. Let number of heads be one. We use Q, K, V to denote the projected query, key, and value vectors over all inputs of the transformer. The first n inputs to the transformer are n vectors $[\mathbf{x}_i^T, y_i]^T$ where $y_i = \mathbf{w}_\tau \mathbf{x}_i$ where $\mathbf{w}_\tau \in R^d, \mathbf{x} \in R^d$. The last input is $[\mathbf{x}_{n+1}^T, 0]^T$. Assume each $\mathbf{x}_i$ is sampled from a Gaussian distribution $N(0, \sigma^2 I_d)$ where I_d denotes a d dimensional identity matrix. Show the value of the parameters W_k, W_Q, W_v that

can cause the last coordinate of $(Q \times K^T)V$ to match the ordinary least square solution $\mathbf{w}_{\text{OLS}}\mathbf{x}_{n+1}$ where $\mathbf{w}_{\text{OLS}}$ denotes the parameters learned with the first n examples forming the training set. Assume that the training size is large enough so that $X^TX = n\sigma^2 I_d$ where X is a $n \times d$ sized matrix denoting the training dataset. The parameters you specify are only allowed to be a function of n, σ^2, and not depend on $\mathbf{w}_\tau$.

This example will show that transformers can learn new functions on-the-fly, a capability formally called in-context-learning.

5. Show how you can express the bitwise XOR of n bits using a linear RNN with these equations:

$$\mathbf{h}_t = \text{ReLU}(W\mathbf{h}_{t-1} + Ux_t + \mathbf{b})$$
$$o_t = V\mathbf{h}_t + c$$

[Hint: Use the solution for representing XOR using feedforward networks]